



Security Auditing

Lesson#9

Endpoint Forensics – Memory & Volatility3

School Year 2025/2026

Master in Informatics Security (MSI) / Master in Informatics Engineering (MEI)

Informatics Engineering Department

University of Coimbra



Presentation Agenda

- The live state – understanding volatile artifacts
- Acquisition – capturing memory images
- The Volatility3 tool – framework architecture
- Preparation for the Lab session – hands-on analysis guide



Part 1

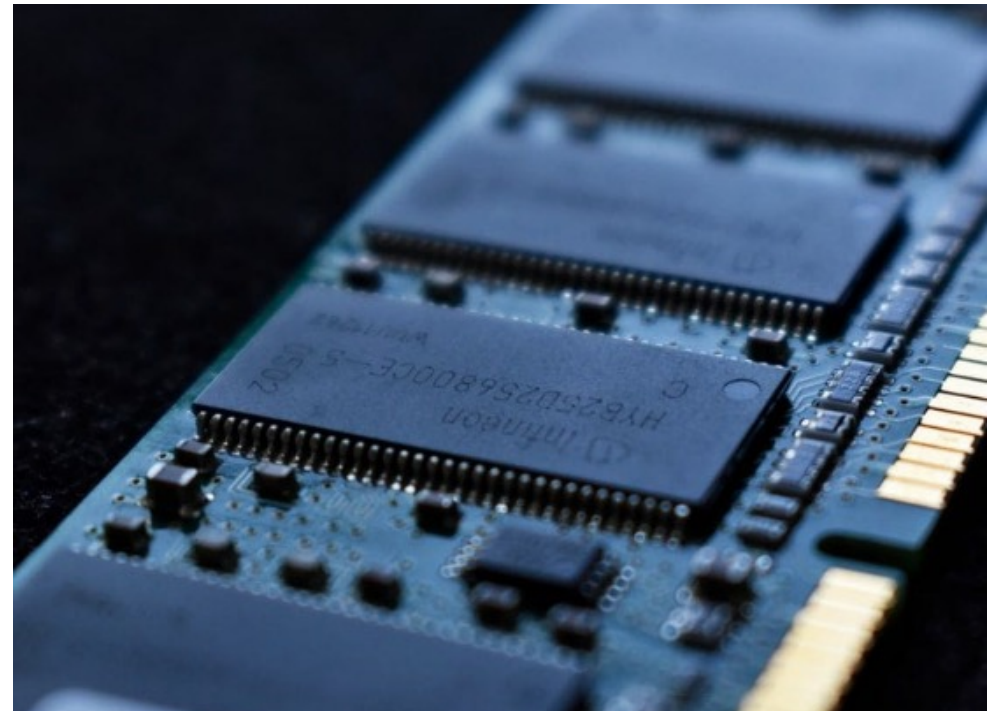
The Live Forensics Ecosystem

Capturing the Ephemeral Crime Scene

The Static Analysis Blind Spot *(recall from last week)*

90% of modern malware resides purely in RAM

- static analysis misses "Fileless" threats.
- live response captures transient network states.
- encryption keys often exist only in memory.





RAM: the new Smoking Gun *(recap from last week)*

What lives in RAM:

- plaintext passwords and decrypted keys.
- unsaved documents (Word/Notepad).
- malware processes hiding from the OS.
- recent CLI commands.

Malware has evolved. “Fileless” malware doesn't sit on the disk where an antivirus can scan it. It lives in the memory space of a legitimate process (like explorer.exe). RAM is a dump of active data structures. Tools such as Volatility 3 can “carve” these structures out of a raw memory dump.



The Order of Volatility (RFC 3227*) *(recap from last week)*

The Golden Rule: collect data from the most volatile to the least volatile.

The hierarchy:

- Registers & Cache (nanoseconds)
- RAM (the “State of Play”)
- Network State (current connections/ARP cache)
- Disk/Physical Media (persistent storage)

In Computer Science, we think of memory hierarchy in terms of speed.

In Forensics, we think of it in terms of survivability. If you pull the power plug, you lose the top three layers instantly. For an auditor, this means that if your incident response plan starts with “Reboot the server”, you are actively destroying evidence.

* <https://www.rfc-editor.org/rfc/rfc3227.html>

The Order of Volatility (RFC 3227) *(recall from last week)*

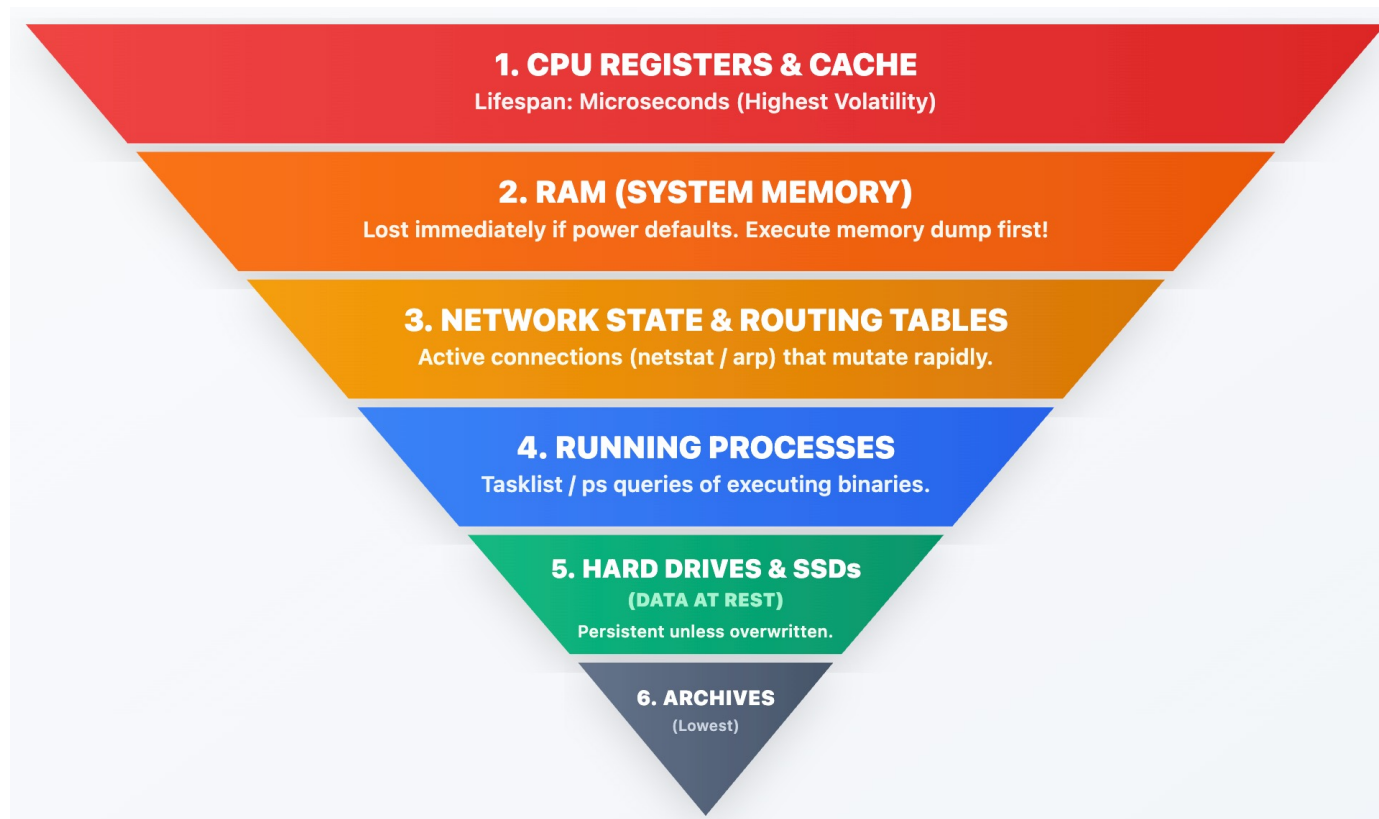


Image source: <https://perfectnotes.org/images/cybersecurity/Cyber%20Forensics/Order%20of%20Volatility.svg>



Part 2

Memory Acquisition Techniques

Tools and Hardware for High-Integrity Dumps



Types of memory artifacts

- **Physical RAM (*.raw, *.bin):**
Bit-for-bit copy of the physical address space.
- **Hibernation File (e.g. hiberfil.sys):**
Compressed snapshot of RAM saved to disk when a system sleeps.
- **Page File (e.g. pagefile.sys):**
Virtual memory stored on disk to free up physical RAM.
- **Crash Dumps (*.dmp):**
Memory state saved by the OS after a kernel panic or Blue Screen of Death (BSOD).



Technical Triage Tools *(recall from last week)*

“Remote/Fast” forensics

- Traditional:** physically seize the drive (high friction).
- Modern (Readiness):** pre-deployed “Triage Agents”
- Tools:** Velociraptor, KAPE, EDR-integrated tools, etc.
- Benefit:** collect critical artifacts (RAM, Registry) in minutes, over the network.

We need to be efficient. If you have 500 workstations in 5 different cities, you can't fly a tech to each one. Forensic readiness means having an “agent” already sitting there, waiting for your command to securely upload a memory dump.



Live vs. Dead Acquisition *(recall from last week)*

Dead Box (static): imaging a powered-off drive (safest, easiest to hash).

Live Acquisition (dynamic): Imaging while the OS is running.

The trade-off: live acquisition alters the system (minimal footprint), but it is the only way to get encryption keys and RAM.

You will hear purists say “always pull the plug”, but if the drive is encrypted with BitLocker or FileVault, and you don’t have the recovery key, that drive is now a paperweight. Live Acquisitions may capture the keys while they are “unlocked” in memory.

Software-based RAM Capture (Windows)

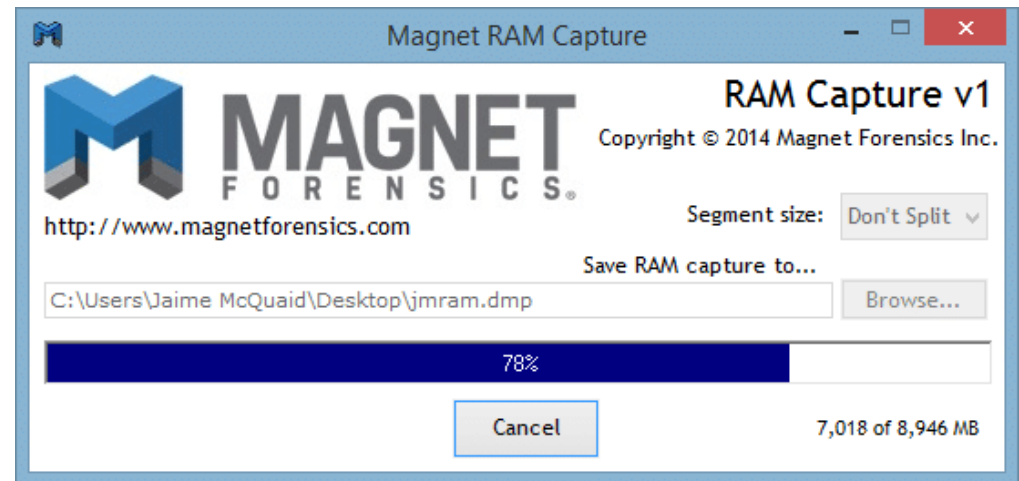
Magnet RAM capture

<https://www.magnetforensics.com/resources/magnet-ram-capture/>

- minimal kernel footprint.
- simple GUI/CLI for first responders.
- supports raw image output.
- processing of raw capture is outside the scope of the tool.
- still popular despite its age & the availability of new versions (e.g. Magnet DumpIT for Windows)

Short introduction:

<https://www.magnetforensics.com/blog/acquiring-memory-with-magnet-ram-capture/>





Software-based RAM Capture (Linux & Mac)

LiME (Linux Memory Extractor)

<https://github.com/jtsylve/LiME>

- a kernel module (LKM) for full physical memory acquisition.
- high reliability for non-standard kernels.

Microsoft AVML

<https://github.com/microsoft/avml>

- modern acquisition tool written in Rust.
- optimized for cloud triage and Linux server fleets.



Hardware-based RAM Capture (DMA)

Hardware-level RAM acquisition techniques that interface directly with physical memory, often used in digital forensics and hardware security auditing.

Hardware solutions:

FPGA-based PCIe boards, Thunderbolt-to-PCIe adapters, and USB-C devices.

PCILeech:

Widely used framework for memory acquisition and hardware-level interaction.

MemProcFS:

Presents physical memory as a virtual file system to facilitate analysis.

In-house DMA toolkits:

Custom scripts often developed for specific forensic or research environments.



Hardware-based RAM Capture (DMA) *(cont'd)*

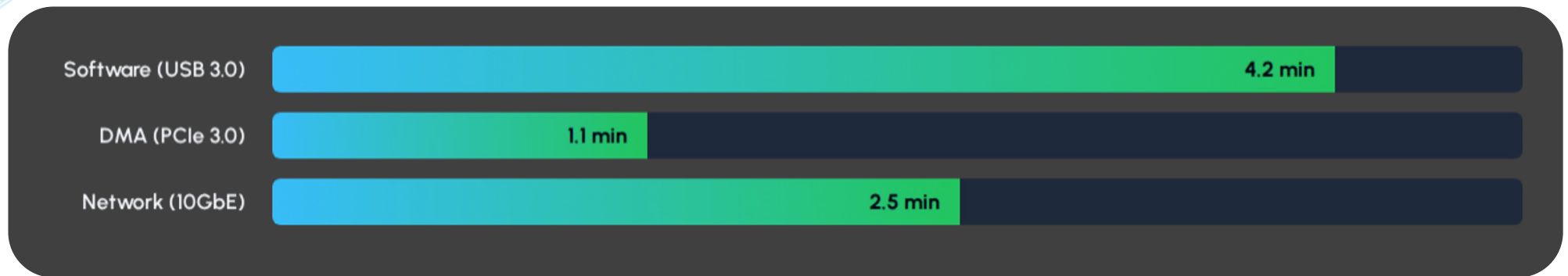
Advantages:

- **Minimal Software Footprint:**
hardware level, reducing the interaction with the target operating system's software state.
- **Reliability:**
can access memory even when the operating system is unresponsive or locked.
- **Speed:**
provide high-speed access to physical memory via direct bus communication.

Disadvantages and Limitations

- **Hardware Requirements:**
require specific physical hardware and often physical access to the target device.
- **Input-Output Memory Management Unit (IOMMU) Mitigations:**
hardware security features like IOMMU or VT-d can block unauthorized DMA access.
- **Risk of System Instability:**
DMA can occasionally cause system crashes or data corruption if not handled carefully.

Comparison of Acquisition Time



Values based on 16GB RAM full physical acquisition.



Hashing and Metadata Integrity

- **Cryptographic Hashing:**
compute SHA-256 immediately after acquisition to ensure zero changes.
- **Time Offsets:**
record system BIOS time vs. real-world time to align logs.
- **Documentation:**
note physical hardware specs and tool versioning used.



Part 3

Volatility3

A framework for processing memory forensics



Evolution of the framework

“Volatility 3 is not just an update; it's a total rewrite in Python 3, removing the friction of legacy profiles for a symbol-driven future”

- content provided in these materials refers to Volatility3, even though Volatility2 is still in use (the way the tools work is slightly different).

Symbols vs. Profiles (i.e., how to interpret raw memory for different OS's)

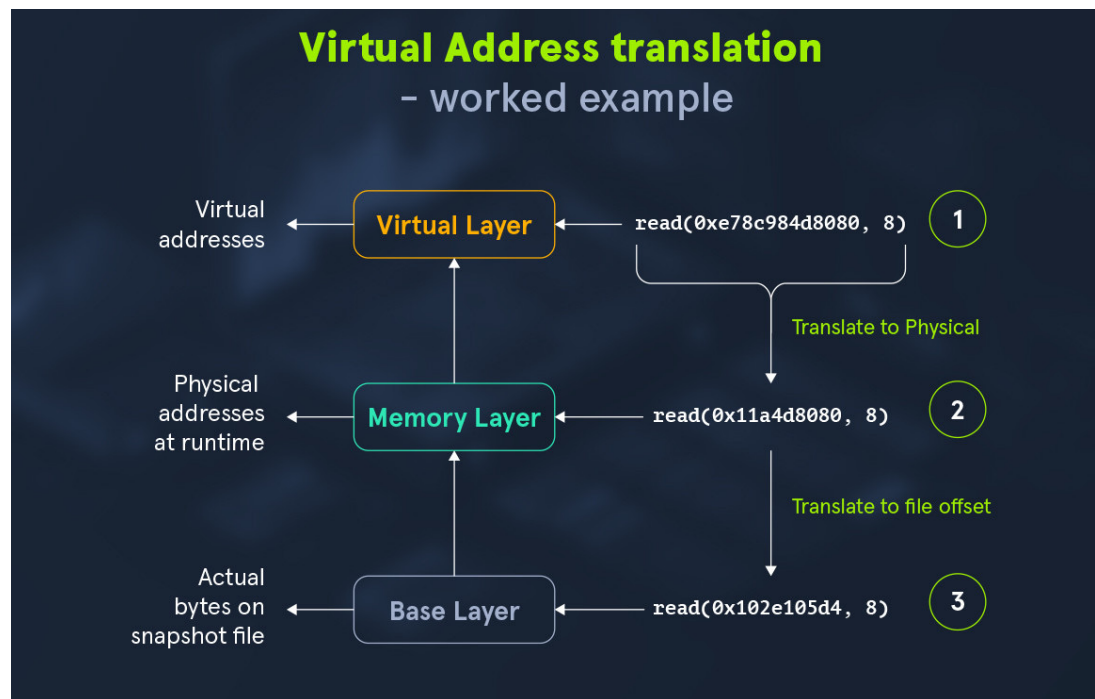
- **Volatility2 required “Profiles” (KDBG blocks)**
needed to be pre-generated.
analysts often struggled with missing profiles for new Windows patches.
- **Volatility3 uses “Symbols” (PDB files).**
downloaded on-the-fly from symbol servers.
profile management becomes transparent to the user.

Intermediate Symbol Profile (ISF)

Volatility 3 translates debugging symbols into a JSON-based format called ISF.

This allows to:

- identify structure offsets automatically.
- support Windows, Linux, and Mac OS kernels.
- remain static and library-independent.



Nice tutorial and image source:

<https://www.hackthebox.com/blog/how-to-create-linux-symbol-tables-volatility>



Automagic Components

Automatically handle setup, configuration & analysis tasks that were previously manual or required user-defined profiles in Volatility 2.

Primary role: eliminates the need for users to manually identify and specify operating system profiles, kernel memory offsets, or symbol tables.



1. CONFIGURATION

The "Automagic" system scans for layers and identifies the OS without manual input.



2. LAYERS

Address translation happens across physical and virtual layers (DTB/CR3 mapping).



3. PLUGINS

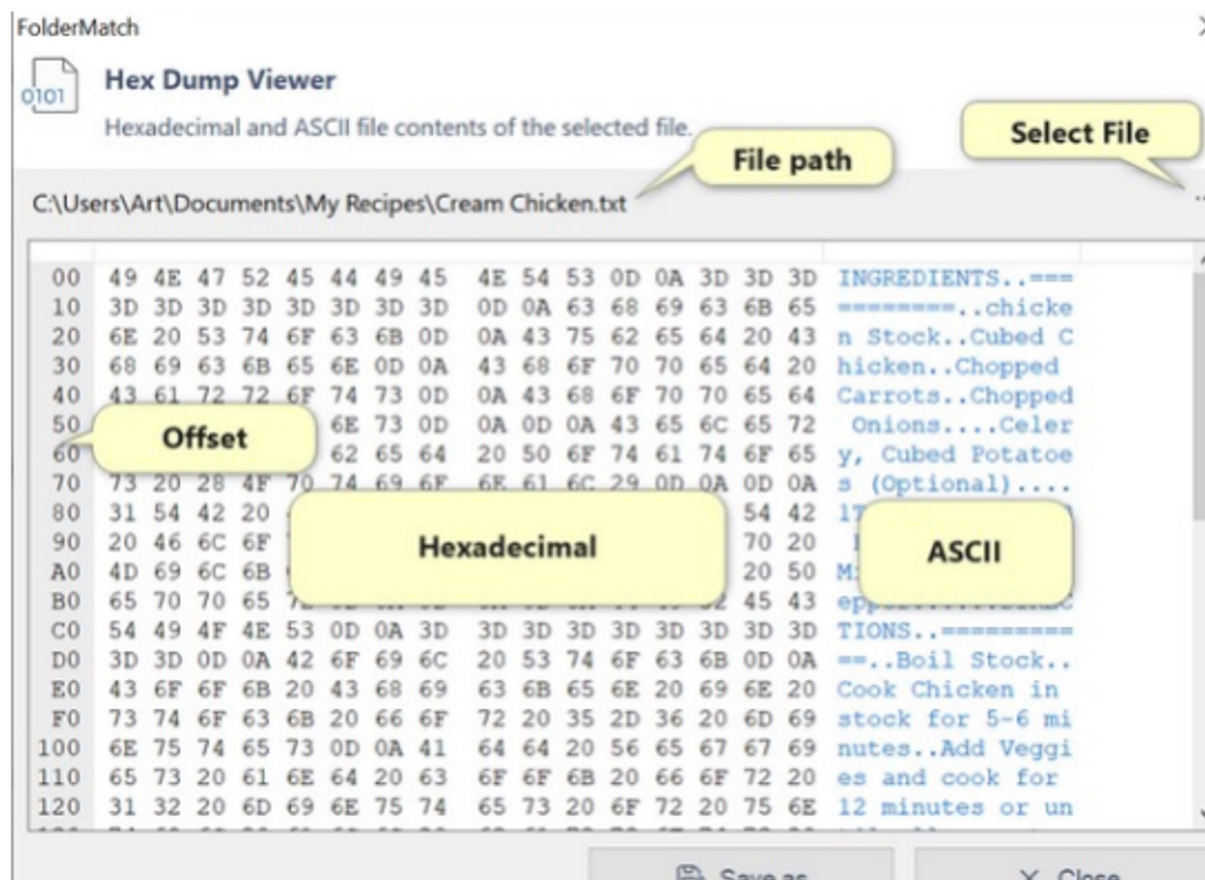
The plugin logic remains clean, calling the context to read objects via symbols.

Managing Symbol Tables

Windows symbols are downloaded automatically to volatility3/symbols directory.

If your lab environment is offline, you must manually populate this cache.

Use the `pdbconv.py` tool to convert Windows PDBs to Volatility ISFs.





Installation and Configuration

More direct approach (Linux machine and Python 3.8+ installed):

- *pip install volatility3*
- or the alternative methods mentioned here: <https://github.com/volatilityfoundation/volatility3>
- don't forget the symbol tables.

Going Kali-style... install Linux distributions preloaded with Volatility3, such as:

- SIFT Workstation, <https://www.sans.org/tools/sift-workstation>
- CAINE <https://www.caine-live.net/>

In case you have a Mac ARM64 (Apple M1/M2/M3):

- you may pick an Ubuntu VM and then install Volatility3 (or SIFT, in case you have time) from there.
- alternatively, you may download the UTM Ubuntu VM pre-installed with SIFT and available at *UC Student* (login/password: *ubuntu/ubuntu*)



Part 4

Mastering the Volatility3 Command Line

Syntax, namespacing, and automation



Plugin namespacing syntax

Structure: [OS].[Plugin]:

\$ python3 vol.py -f **dump.raw** windows.pslist (*Windows*)

\$ python3 vol.py -f **dump.raw** linux.pslist (*Linux*)

\$ python3 vol.py -f **dump.raw** mac.pslist (*MacOS*)

\$ python3 vol.py -f *[/path/to/memory_dump_file.raw] [plugin_name]*

When you run this command, Volatility maps the file into its engine to simulate the state of the RAM at the moment it was captured.

If you are analyzing a memory dump from a virtual machine (like a .vmem file from VMware), you still use the -f flag to point directly to that virtual memory file.



Volatility output formats

For large scale investigations, use structured output:

```
$ python3 vol.py -f dump.raw windows.pslist -r [json | csv | quick...]
```

JSON output allows you to ingest results directly into ELK, Splunk, or custom Python analysis scripts.



Part 4

Core Windows Analysis

Essential plugins for the Lab



Identification with Windows Info

**The first plugin to run...
identifies the target OS version, kernel offsets, and CPU count**

```
$ python3 vol.py -f mem.raw windows.info
```

Key Artifact: The “Is64Bit” flag and “Symbol Server” status.



Process Analysis: pslist, pstree, PsScan

pslist:

- flat list of EPROCESS structures
- does not detect hidden or unlinked processes (*psscan* can do that)

pstree:

- visualizes parent-child hierarchy (identifies orphaned processes)

psscan:

- carves memory for terminated processes hidden from the active list

<https://junosou-60595.medium.com/difference-between-pslist-and-psscan-edc9b932ea29>



Networking: netscan & netstat (from RAM)

Plugins *windows.netscan* and *windows.netstat*:

```
$ python3 vol.py -f /path/to/memory.dmp windows.netscan
```

```
$ python3 vol.py -f /path/to/memory.dmp windows.netstat
```

Extracting Active Sockets:

- list TCP/UDP connections, including local/remote IP and the associated PIDs

Look for:

High-number ports connecting to external IPs, common for Command & Control (C2)



Analysing DLLs and Handles

Plugin	Forensic Use Case
windows.dlllist	Identify malicious libraries loaded by a specific PID.
windows.handles	Discover open files, mutexes, and registry keys used by an attacker.
windows.modules	List loaded kernel drivers (detect rootkits).



Registry Analysis (from RAM data)

Access the hive files (e.g. System, Software) without needing the disk image.

Plug-in windows.registry.hivelist:

windows.registry.printkey --key "Software\Microsoft\Windows\CurrentVersion\Run"

Enumerates Windows startup programs, identifying potential malware persistence mechanisms.
Displays all autorun entries in the Registry hive without needing to extract the hive file from memory.

Example Usage:

vol -f memory.dmp windows.registry.printkey -key "Software\Microsoft\Windows\CurrentVersion\Run"



Part 4

Malware Hunting with Volatility3

Detecting Code Injection and Stealthy Threats



Detecting Malware with “windows.malfind”

Identifies potential code injection by scanning for:

- RWX (Read-Write-Execute) permissions.
- Memory segments not backed by a file on disk.
- Characteristic shellcode headers (e.g., MZ).

Code injection indicators:

- Unbacked Mmemory: executable code existing in a memory heap or stack.
- PE Headers: the presence of an “MZ” string at the start of a memory segment. (*why MZ?*)
- Hollow processes: legitimate process names (e.g. “svchost.exe”) with suspicious child activity. (*what does this mean?*)



VAD – Virtual Address Description

What is VAD?

- The kernel's method of managing a process's virtual memory layout.
- It's an AVL tree structure.

VAD analysis:

- by comparing the VAD to the loaded DLL list,
Volatility can detect when code is “unhooked” or “hollowed”.



Dumping Malicious Artifacts

Once a malicious PID is identified, it is possible to extract cached files from a Windows memory image, which allows recovering malicious executables or configuration files, for reverse engineering:

```
windows.dumpfiles.DumpFiles --pid [PID]
```

```
windows.memmap.MemMap --pid [PID] -dump
```

```
$ python3 vol.py -f <memory_dump_file> windows.dumpfiles
```

Extracted files can then be analyzed in Ghidra, IDA Pro, or uploaded to VirusTotal.



Advanced Triage: MemProcFS

Instead of complex command-line arguments typical of traditional tools like Volatility, *MemProcFS* turns memory into a navigable file structure (sys/proc, sys/net, sys/registry). It mounts memory as a virtual drive.

Allows standard Windows tools to browse RAM contents:

- browse process memory via folders.
- view registry hives as files.
- instant timeline extraction.

It may connect directly to live memory.



Linux Forensics Analysis Basics

linux.pslist:

- lists Linux processes

linux.lsof:

- lists open files by process

linux.check_syscall:

- detects hijacked system calls (rootkit hunting)



Tracking User Activity using RAM (examples)



CREDENTIALS

Extract cleartext passwords from LSASS memory dumps or session tokens.



WEB BROWSING

Recover URL history, form data, and active chat sessions from browser processes.



CLIPBOARD

Extract recent clipboard contents often containing passwords or IP addresses.



Q&A

Ready for the Lab?